SÉANCE 2 · LIVRET ÉTUDIANT

L’hexagone, ports et adapters

Mettre le métier au centre et brancher la technique autour, par injection de dépendances

Élément	Détail
Module	Architecture Hexagonale & DDD (A2627_4671) - Bachelor 3 Informatique - Développement
Séance	Séance 2 - lundi 19 octobre 2026, 14h-17h (3 h, face à face pédagogique)
Fil rouge	Le Fournil de Serris, boulangerie de quartier qui lance son Click & Collect (fictif)
Dossier de départ	`boulangerie-click-collect-seance2` : l’architecture en couches de la séance 1, complète
Prérequis	Séance 1 (couches, DIP). Interfaces TypeScript, `async`/`await`.
TP de la séance	Pas de TP à rendre : les étapes 6 et 7 sont des exercices guidés, repris au TP1 demain.

> Ce livret se suit pendant la séance. Si vous décrochez de l’oral, pas de panique : chaque étape est écrite pour que vous puissiez la refaire seul. Repartez du kit de la séance 2 même si votre découpage du matin est terminé : tout le monde part du même code.

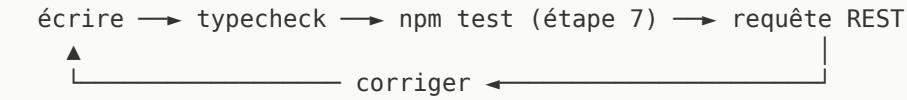
Comment utiliser ce livret

La séance compte **7 étapes**. Les étapes 1 à 5 se codent ensemble (vous tapez en même temps) ; les étapes 6 et 7 se font seul, en exercice guidé.

Rubrique	Ce qu'elle contient	Logique
À quoi ça sert	Le problème d'architecture, avant le code.	EXPLIQUER
Ce que vous tapez	Les fichiers à créer, avec leur chemin exact.	MODIFIER
Ce que vous devez voir	Le résultat : API identique, `typecheck` propre, tests verts.	TESTER
Le piège	L'erreur classique, et comment la reconnaître.	-
À vous	Un exercice court avec un « pourquoi ».	MODIFIER + TESTER

La boucle à répéter tout le temps

À chaque fichier créé



> **Et l'IA ?**
> Rappel : l'IA vous explique une erreur de typage ou un concept, elle n'écrit pas vos ports. Demain, le TP1 est sans IA : c'est maintenant qu'il faut comprendre chaque ligne.

Ce que contient ce livret

- Étapes 1 à 5 : l'hexagone pour le cas d'usage « Passer une commande ».
- Étapes 6 et 7 : un second adapter (mémoire) et des tests unitaires sans base ni serveur.
- Mémo express : vocabulaire de l'hexagone, règle des dépendances, commandes.

Avant de commencer : checklist

Vérification	Comment	OK ?
Kit S2 décompressé et ouvert	`boulangerie-click-collect-seance2` dans Documents, `npm install` fait	☐
API en route	`npm run dev`, puis `GET /produits` répond	☐
Dépôt Git initialisé	`git init`, `git add .`, `git commit -m "Kit séance 2"`	☐

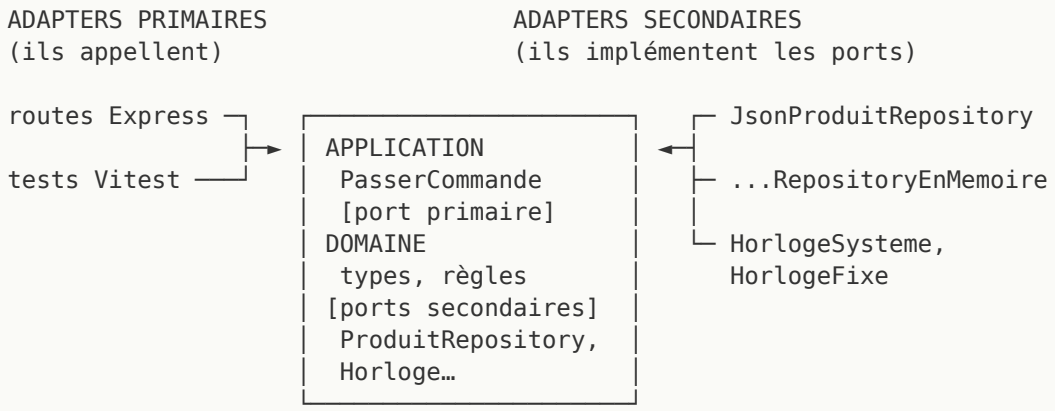
Étape 1 - L'hexagone en un schéma

Durée indicative : 20 min

À quoi ça sert · EXPLIQUER

L'**architecture hexagonale** (ou *ports and adapters*) met le métier au centre. Tout ce qui est technique est un **adapter** branché sur une **prise** définie par le métier : le **port**. La forme hexagonale n'a pas de sens particulier : elle laisse simplement de la place pour dessiner plusieurs prises.

À retenir : toutes les dépendances pointent VERS le centre



Mot	Définition	Dans le Fournil
Domaine	Le vocabulaire et les règles du métier, sans aucun import technique	`src/domain/`
Cas d'usage	Une action que le métier sait faire (port primaire)	`PasserCommande`
Port secondaire	Interface définie par le métier pour ce dont il a besoin	`ProduitRepository`, `Horloge`
Adapter primaire	Traduit une entrée (HTTP, CLI, test) en appel de cas d'usage	`commandeRoutes.ts`
Adapter secondaire	Implémente un port avec une technique précise	`JsonProduitRepository`
Point de composition	Le seul endroit qui crée les classes concrètes et les branche	`composition.ts`

Ce que vous tapez · MODIFIER

Arborescence cible (créez les dossiers vides)

```
src/
├── domain/                 types.ts, ErreurMetier.ts
├── application/            PasserCommande.ts, ConsulterCatalogue.ts,
│   │                       ConsulterCommande.ts
│   └── ports/              ProduitRepository.ts, CommandeRepository.ts,
│                           Horloge.ts
├── adapters/
│   ├── primaires/http/     commandeRoutes.ts, erreurs.ts
│   └── secondaires/
│       ├── json/           JsonFileDb.ts, JsonProduitRepository.ts,
│       │                   JsonCommandeRepository.ts
│       ├── memoire/        ...EnMemoire.ts, HorlogeFixe.ts
│       └── systeme/        HorlogeSysteme.ts
├── legacy/                 routesHistoriques.ts (pas encore migré)
├── composition.ts
└── main.ts
tests/                      PasserCommande.test.ts
```

Ce que vous devez voir · TESTER

Ce que vous devez voir

L'arborescence créée. Vous savez dire, pour chaque dossier, s'il a le droit d'importer `express` ou `node:fs` (seulement `adapters/` et `composition.ts`).

Le piège

Confondre couches et hexagone. En couches, le métier importe l'infrastructure. Dans l'hexagone, c'est l'inverse : l'infrastructure importe le métier pour implémenter ses ports.

À vous

Classez : une commande CLI `npm run commande` (primaire ou secondaire ?), un envoi d'email, un test Vitest, une API de paiement externe.

Étape 2 - Isoler le domaine

Durée indicative : 15 min

À quoi ça sert · EXPLIQUER

Le domaine donne des **noms et des types** à ce que manipule la boulangerie. En couches, tout était `any`. Ici, aucune ligne ne doit importer une bibliothèque technique : c'est ce qui permettra de le tester et de le réutiliser partout.

Ce que vous tapez · MODIFIER

src/domain/types.ts

```
// Le domaine : le vocabulaire de la boulangerie. Aucun import technique ici.
export type StatutCommande = 'EN_ATTENTE' | 'PRETE' | 'RETIREE' | 'ANNULEE';

export interface Produit {
  id: string;
  nom: string;
  prix: number; // en centimes
  stock: number;
}

export interface LigneCommande {
  produitId: string;
  quantite: number;
}

export interface Commande {
  id: string;
  client: string;
  telephone: string;
  lignes: LigneCommande[];
  retrait: string; // date et heure locales, ex. 2026-10-20T08:30:00
  total: number; // en centimes
  statut: StatutCommande;
  creeLe: string;
}
```

src/domain/ErreurMetier.ts

```
export type CodeErreur = 'INVALIDE' | 'INTROUVABLE' | 'CONFLIT';

export class ErreurMetier extends Error {
  constructor(message: string, readonly code: CodeErreur) {
    super(message);
  }
}
```

Ce que vous devez voir · TESTER

> **Ce que vous devez voir**
>
> `npm run typecheck` sans erreur. Une recherche `import` dans `src/domain/` ne trouve aucune bibliothèque.

Le piège

Le type du framework qui s'invite. `import type { Request } from 'express'` dans le domaine pour typer une demande : c'est déjà une dépendance vers Express. Le domaine définit ses propres types.

À vous

Pourquoi `retrait` est-il une chaîne « locale » (`2026-10-20T08:30:00`, sans `Z`) et pas une date UTC ? Pensez à la règle « entre 7h et 19h30 ».

Étape 3 - Écrire les ports secondaires

Durée indicative : 20 min

À quoi ça sert · EXPLIQUER

Reprenez la liste des besoins écrite à la fin de la séance 1. Chaque besoin devient une méthode d'**interface**, écrite dans la langue du métier, rangée **dans** l'application : c'est le métier qui possède ses ports (DIP). Les méthodes sont `async` : demain, une vraie base répondra de façon asynchrone.

Ce que vous tapez · MODIFIER

src/application/ports/ProduitRepository.ts

```
import type { Produit } from '../../domain/types';

// Port secondaire (sortie) : ce dont le métier a besoin, pas comment c'est stocké.
export interface ProduitRepository {
  tous(): Promise<Produit[]>;
  trouverParId(id: string): Promise<Produit | undefined>;
  enregistrer(produit: Produit): Promise<void>;
}
```

src/application/ports/CommandeRepository.ts

```
import type { Commande } from '../../domain/types';

export interface CommandeRepository {
  prochainId(): Promise<string>;
  trouverParId(id: string): Promise<Commande | undefined>;
  enregistrer(commande: Commande): Promise<void>;
}
```

src/application/ports/Horloge.ts

```
// Le temps est une dépendance comme une autre : on l'injecte pour pouvoir le tester.
export interface Horloge {
  maintenant(): Date;
}
```

Ce que vous devez voir · TESTER

Ce que vous devez voir

Trois interfaces, aucune implémentation, aucun mot technique (fichier, JSON, SQL) dans les noms de méthode.

Le piège

Le port qui parle technique. `executerRequete(sql: string)` ou `lireFichier()` font fuir la technique dans le métier : impossible d'écrire un adapter en mémoire sans simuler du SQL.

À vous

Pourquoi l'heure courante mérite-t-elle un port, alors que `Math.round` n'en a pas besoin ?

Étape 4 - Le cas d’usage PasserCommande

Durée indicative : 25 min

À quoi ça sert · EXPLIQUER

Le cas d’usage est le **port primaire** : la porte d’entrée du métier. Il reçoit ses dépendances **par le constructeur** (*injection de dépendances*) et ne connaît que des interfaces. Les règles sont celles de la séance 1, à l’identique ; seule change la provenance des données et de l’heure.

Ce que vous tapez · MODIFIER

src/application/PasserCommande.ts

```
// Cas d'usage (port primaire) : « Passer une commande ».
// Il ne connaît que des interfaces (ports) : aucun fichier, aucun Express.
import type { Commande, LigneCommande } from '../domain/types';
import { ErreurMetier } from '../domain/ErreurMetier';
import type { ProduitRepository } from './ports/ProduitRepository';
import type { CommandeRepository } from './ports/CommandeRepository';
import type { Horloge } from './ports/Horloge';

export interface DemandeCommande {
  client: string;
  telephone: string;
  lignes: LigneCommande[];
  retrait: string;
}

const DEUX_HEURES = 2 * 60 * 60 * 1000;

export class PasserCommande {
  constructor(
    private readonly produits: ProduitRepository,
    private readonly commandes: CommandeRepository,
    private readonly horloge: Horloge,
  ) {}

  async executer(demande: DemandeCommande): Promise<Commande> {
    const { client, telephone, lignes, retrait } = demande;
    if (!client || !telephone) throw new ErreurMetier('Nom et téléphone
obligatoires', 'INVALIDE');
    if (!lignes || lignes.length === 0) throw new ErreurMetier('Commande vide',
'INVALIDE');

    let total = 0;
    for (const ligne of lignes) {
      const produit = await this.produits.trouverParId(ligne.produitId);
      if (!produit) throw new ErreurMetier(`Produit inconnu : ${ligne.produitId}`,
'INTROUVABLE');
      if (ligne.quantite < 1 || ligne.quantite > 20) {
        throw new ErreurMetier('Quantité entre 1 et 20 par produit', 'INVALIDE');
      }
      if (produit.stock < ligne.quantite) {
        throw new ErreurMetier(`Stock insuffisant : ${produit.nom}`, 'CONFLIT');
      }
      total += produit.prix * ligne.quantite;
    }

    const maintenant = this.horloge.maintenant();
    const dateRetrait = new Date(retrait);
    if (dateRetrait.getTime() - maintenant.getTime() < DEUX_HEURES) {
      throw new ErreurMetier('Retrait possible au plus tôt 2 h après la commande',
'INVALIDE');
    }
    const minutes = dateRetrait.getHours() * 60 + dateRetrait.getMinutes();
    if (minutes < 7 * 60 || minutes > 19 * 60 + 30) {
      throw new ErreurMetier('Retrait entre 7h et 19h30', 'INVALIDE');
    }

    if (total >= 3000) total = Math.round(total * 0.9); // remise fidélité -10 % dès
30 €
```

```
    for (const ligne of lignes) {
      const produit = (await this.produits.trouverParId(ligne.produitId))!;
      await this.produits.enregistrer({ ...produit, stock: produit.stock -
ligne.quantite });
    }

    const commande: Commande = {
      id: await this.commandes.prochainId(),
      client,
      telephone,
      lignes,
      retrait,
      total,
      statut: 'EN_ATTENTE',
      creeLe: maintenant.toISOString(),
    };
    await this.commandes.enregistrer(commande);
    return commande;
  }
}
```

Sur le même modèle, écrivez `ConsulterCatalogue` (renvoie `produits.tous()`) et `ConsulterCommande` (lève `INTROUVABLE` si besoin).

Ce que vous devez voir · TESTER

Ce que vous devez voir

`npm run typecheck` propre. Les seuls `import` de `src/application/` pointent vers `../domain/` ou `./ports/`.

Le piège

Le `new` caché. `private produits = new JsonProduitRepository(…)` dans le cas d'usage : il recrée la dépendance de la séance 1. On reçoit, on ne fabrique pas.

`Date.now()` oublié. Une seule ligne avec `new Date()` (sans argument) et vos tests redeviendront dépendants de l'heure.

À vous

Dans `PasserCommande`, relisez la ligne `creeLe`. D'où vient l'heure ? Que se passerait-il en test si on avait gardé `new Date().toISOString()` ?

Étape 5 - Les adapters JSON et le point de composition

Durée indicative : 30 min

À quoi ça sert · EXPLIQUER

Les adapters secondaires **implémentent** les ports avec une technique. Le point de composition est le seul fichier qui connaît toutes les classes concrètes : il les crée et les branche. La route `prete` n'est pas encore migrée : elle part dans `src/legacy/`, on la traitera au TP1.

Ce que vous tapez · MODIFIER

src/adapters/secondaires/json/JsonFileDb.ts

```
import fs from 'node:fs';

// Adapter technique : lit et écrit le fichier JSON qui sert de base.
export class JsonFileDb {
  constructor(private readonly chemin: string) {}

  lire(): any {
    return JSON.parse(fs.readFileSync(this.chemin, 'utf-8'));
  }

  ecrire(db: any): void {
    fs.writeFileSync(this.chemin, JSON.stringify(db, null, 2));
  }
}
```

src/adapters/secondaires/json/JsonProduitRepository.ts

```
import type { Produit } from '../../../domain/types';
import type { ProduitRepository } from
'../../../application/ports/ProduitRepository';
import type { JsonFileDb } from './JsonFileDb';

// Adapter secondaire : implémente le port ProduitRepository avec un fichier JSON.
export class JsonProduitRepository implements ProduitRepository {
  constructor(private readonly db: JsonFileDb) {}

  async tous(): Promise<Produit[]> {
    return this.db.lire().produits;
  }

  async trouverParId(id: string): Promise<Produit | undefined> {
    return this.db.lire().produits.find((p: Produit) => p.id === id);
  }

  async enregistrer(produit: Produit): Promise<void> {
    const data = this.db.lire();
    data.produits = data.produits.map((p: Produit) => (p.id === produit.id ?
produit : p));
    this.db.ecrire(data);
  }
}
```

src/composition.ts

```
// Point de composition : le SEUL endroit qui connaît toutes les classes concrètes.
// On y branche les adapters sur les ports (injection de dépendances).
import express from 'express';
import type { Horloge } from './application/ports/Horloge';
import { PasserCommande } from './application/PasserCommande';
import { ConsulterCatalogue } from './application/ConsulterCatalogue';
import { ConsulterCommande } from './application/ConsulterCommande';
import { JsonFileDb } from './adapters/secondaires/json/JsonFileDb';
import { JsonProduitRepository } from
'./adapters/secondaires/json/JsonProduitRepository';
import { JsonCommandeRepository } from
'./adapters/secondaires/json/JsonCommandeRepository';
import { HorlogeSysteme } from './adapters/secondaires/systeme/HorlogeSysteme';
import { commandeRoutes } from './adapters/primaires/http/commandeRoutes';
import { routesHistoriques } from './legacy/routesHistoriques';

export interface Options {
  horloge?: Horloge;
}

export function creerApp(cheminDb: string, options: Options = {}) {
  const horloge = options.horloge ?? new HorlogeSysteme();
  const db = new JsonFileDb(cheminDb);
  const produits = new JsonProduitRepository(db);
  const commandes = new JsonCommandeRepository(db);

  const app = express();
  app.use(express.json());
  app.use(
    commandeRoutes({
      passerCommande: new PasserCommande(produits, commandes, horloge),
      consulterCatalogue: new ConsulterCatalogue(produits),
      consulterCommande: new ConsulterCommande(commandes),
    }),
  );
  app.use(routesHistoriques(db));
  return app;
}
```

src/main.ts

```
import { creerApp } from './composition';

const app = creerApp(process.env.DB_FILE ?? 'data/db.json');
app.listen(3000, () => console.log('Fournil de Serris sur http://localhost:3000'));
```

La suite est à vous : `JsonCommandeRepository` (sur le modèle du produit), `HorlogeSysteme` (renvoie `new Date()` : c'est le seul endroit autorisé), `erreurs.ts` et `commandeRoutes.ts` (l'adapter primaire reçoit les cas d'usage en paramètre), et `legacy/routesHistoriques.ts` (la route `prete` de la séance 1, qui utilise `JsonFileDb`). Supprimez ensuite `presentation/`, `metier/`, `infrastructure/` et `app.ts`.

Ce que vous devez voir · TESTER

> **Ce que vous devez voir**
>
> Les 4 requêtes de `requetes.http` donnent les mêmes réponses qu'avec le kit de départ.
>
> Recherche `new Json` dans `src/` : uniquement dans `composition.ts`.

Le piège

L'adapter instancié dans le domaine. Si `PasserCommande` crée son repository, vous ne pourrez plus le remplacer à l'étape 6. Le point de composition est le seul endroit des `new` d'adapters.

À vous

Commitez (`git commit -m "Hexagone : PasserCommande"`). Puis lancez l'API sur une autre base : `DB_FILE=data/demo.json npm run dev` (copiez d'abord `db.json` en `demo.json` ; sous Windows PowerShell : `$env:DB_FILE="data/demo.json"; npm run dev`). Quel fichier avez-vous modifié pour changer de base ?

Étape 6 - Un second adapter : la mémoire

Durée indicative : 20 min · exercice guidé

À quoi ça sert · EXPLIQUER

Preuve que le métier est indépendant : on lui branche **un autre adapter** sans toucher une ligne du domaine ni de l'application. La mémoire sert aux tests (rapide, isolée). C'est le principe ouvert/fermé et la substitution de Liskov à l'échelle de l'architecture : tout adapter respecte le contrat du port.

Ce que vous tapez · MODIFIER

src/adapters/secondaires/memoire/ProduitRepositoryEnMemoire.ts

```
import type { Produit } from '../../../domain/types';
import type { ProduitRepository } from
'../../../application/ports/ProduitRepository';

// Adapter secondaire pour les tests : tout reste en mémoire.
export class ProduitRepositoryEnMemoire implements ProduitRepository {
  private readonly produits = new Map<string, Produit>();

  constructor(initiaux: Produit[] = []) {
    initiaux.forEach((p) => this.produits.set(p.id, { ...p }));
  }

  async tous(): Promise<Produit[]> {
    return [...this.produits.values()];
  }

  async trouverParId(id: string): Promise<Produit | undefined> {
    return this.produits.get(id);
  }

  async enregistrer(produit: Produit): Promise<void> {
    this.produits.set(produit.id, { ...produit });
  }
}
```

src/adapters/secondaires/memoire/HorlogeFixe.ts

```
import type { Horloge } from '../../../application/ports/Horloge';

// Horloge de test : on choisit l'heure qu'il est.
export class HorlogeFixe implements Horloge {
  constructor(private instant: Date) {}

  maintenant(): Date {
    return this.instant;
  }
}
```

La suite est à vous : `CommandeRepositoryEnMemoire` (même principe, avec `prochainId()` basé sur la taille de la `Map`).

Ce que vous devez voir · TESTER

Ce que vous devez voir

`npm run typecheck` propre ; `git diff --stat` ne montre **aucun** fichier modifié dans `src/domain/` ni `src/application/`.

Le piège

La référence partagée. Stocker `produit` au lieu de `{ ...produit }` : le test peut modifier le stock sans jamais appeler `enregistrer()`, et passer alors que le cas d'usage est faux. Un adapter en mémoire doit se comporter comme une vraie base.

À vous

Quelle différence de comportement subsiste entre l'adapter JSON et l'adapter mémoire ? Est-elle gênante pour les tests ?

Étape 7 - Tester le cas d’usage sans base ni serveur

Durée indicative : 25 min · exercice guidé

À quoi ça sert · EXPLIQUER

Le test assemble lui-même le cas d’usage avec les adapters en mémoire et une horloge fixée : c’est un **adapter primaire** comme un autre. Résultat : des tests rapides, déterministes, qui ne touchent jamais `data/db.json`.

Ce que vous tapez · MODIFIER

tests/PasserCommande.test.ts

```
import { describe, it, expect, beforeEach } from 'vitest';
import { PasserCommande } from '../src/application/PasserCommande';
import { ProduitRepositoryEnMemoire } from
'../src/adapters/secondaires/memoire/ProduitRepositoryEnMemoire';
import { CommandeRepositoryEnMemoire } from
'../src/adapters/secondaires/memoire/CommandeRepositoryEnMemoire';
import { HorlogeFixe } from '../src/adapters/secondaires/memoire/HorlogeFixe';

describe('Passer une commande', () => {
  let produits: ProduitRepositoryEnMemoire;
  let commandes: CommandeRepositoryEnMemoire;
  let passerCommande: PasserCommande;

  beforeEach(() => {
    produits = new ProduitRepositoryEnMemoire([
      { id: 'croissant', nom: 'Croissant au beurre', prix: 120, stock: 40 },
      { id: 'tarte-pommes-6', nom: 'Tarte aux pommes 6 parts', prix: 1800, stock:
4 },
    ]);
    commandes = new CommandeRepositoryEnMemoire();
    const horloge = new HorlogeFixe(new Date('2026-10-19T08:00:00'));
    passerCommande = new PasserCommande(produits, commandes, horloge);
  });

  it('calcule le total et décrémente le stock', async () => {
    const commande = await passerCommande.executer({
      client: 'Camille Martin',
      telephone: '0612345678',
      lignes: [{ produitId: 'croissant', quantite: 6 }],
      retrait: '2026-10-19T11:00:00',
    });

    expect(commande.total).toBe(720);
    expect(commande.statut).toBe('EN_ATTENTE');
    expect((await produits.trouverParId('croissant'))!.stock).toBe(34);
  });

  it('applique -10 % dès 30 €', async () => {
    const commande = await passerCommande.executer({
      client: 'Camille Martin',
      telephone: '0612345678',
      lignes: [{ produitId: 'tarte-pommes-6', quantite: 2 }],
      retrait: '2026-10-19T11:00:00',
    });

    expect(commande.total).toBe(3240); // 36 € - 10 %
  });

  it('refuse un retrait moins de 2 h après la commande', async () => {
    await expect(
      passerCommande.executer({
        client: 'Camille Martin',
        telephone: '0612345678',
        lignes: [{ produitId: 'croissant', quantite: 1 }],
        retrait: '2026-10-19T09:30:00',
      }),
    ).rejects.toThrow('2 h');
  });
});
```

Ce que vous devez voir · TESTER

Ce que vous devez voir

`npm test` : **3 tests passés** en moins d'une seconde, à n'importe quelle heure de la journée.
`git status` : `data/db.json` n'a pas bougé.

Le piège

L'horloge oubliée. Remplacez `HorlogeFixe` par `HorlogeSysteme` : le test du créneau dépend à nouveau de l'heure à laquelle vous le lancez.

À vous

Ajoutez deux tests : « refuse un stock insuffisant » (5 tartes) et « refuse un retrait à 20h ». Vérifiez que le stock de tartes est toujours de 4 après le refus.

Mémo express

Le vocabulaire de l'hexagone

Notion	Sert à	Exemple
Port primaire	Entrée du métier : une action qu'il sait faire	`PasserCommande.executer(demande)`
Port secondaire	Besoin du métier vers l'extérieur, en interface	`ProduitRepository`, `Horloge`
Adapter primaire	Appelle un port primaire depuis une technique	route Express, test Vitest
Adapter secondaire	Implémente un port secondaire	`JsonProduitRepository`, `HorlogeFixe`
Injection de dépendances	Recevoir ses dépendances au lieu de les créer	`constructor(private produits: ProduitRepository)`
Point de composition	Créer et brancher tous les objets concrets	`creerApp()` dans `composition.ts`

La règle des dépendances

Qui a le droit d'importer qui

```
adapters/  ⟶  application/  ⟶  domain/
composition.ts importe tout ; domain/ n'importe rien de technique
```

Les commandes

Action	Commande
Vérifier les types	`npm run typecheck`
Lancer les tests une fois / en continu	`npm test` / `npm run test:watch`
Changer de base de données	`DB_FILE=data/demo.json npm run dev`
Chercher un mot dans le projet	`Ctrl+Maj+F` / `Cmd+Maj+F`

Les 5 réflexes à garder

- Un port se nomme dans la langue du métier et vit dans l'application.
- On reçoit ses dépendances, on ne les crée pas : tous les `new` d'adapters dans `composition.ts`.
- L'heure, l'aléatoire, le réseau et le disque passent par des ports.
- Un adapter en mémoire se comporte comme une vraie base (copies, pas références).
- Un test de cas d'usage tourne sans serveur, sans fichier, à toute heure.